Metaprogramming and Reflection

Academic year 2025-2026 - 1st Session

Exam Requirements for the Practical Part of the Course

Pharo project

Elisa Gonzalez Boix, Aäron Munsters
egonzale@vub.be, amunster@vub.be

The deadline of this assignment is the 8th of May 2026 at 23:59 (Brussels time). Your solution should be uploaded to Canvas as described in Section 3.

1 Context

In this project, you will focus on both structural and behavioural aspects of reflection in Pharo. First, you will extend Pharo's compiler to provide protected modifiers. Second, you will extend Pharo's debugging capabilities to support object-centric debugging. Last, you will extend Pharo's exception handling to support multiple exception handlers.

2 Assignment

2.1 Protected Visibility Modifier

In object-oriented languages, visibility modifiers provide developers means to encapsulate the state and behaviour of objects. As you know, in Smalltalk, instance variables are private, and methods are public. In other object-oriented languages, it is possible to limit the visibility of methods by means of modifiers. For example, in Java, C# and C++, declaring a method as `private` limits the visibility of this method to instances of the same class.

The goal of this extension is to extend Pharo to add support for declaring `protected` methods, which limits the visibility of these methods to self-sends or super-sends from an instance of the class or a subclass. The extension will distinguish syntactically between self-sends an object sends to statically predict the message receiver's type and determine whether invoking protected methods is legal or not. To illustrate the semantics of protected methods, consider the example in Figure 1 consisting of two classes defining public and protected methods.

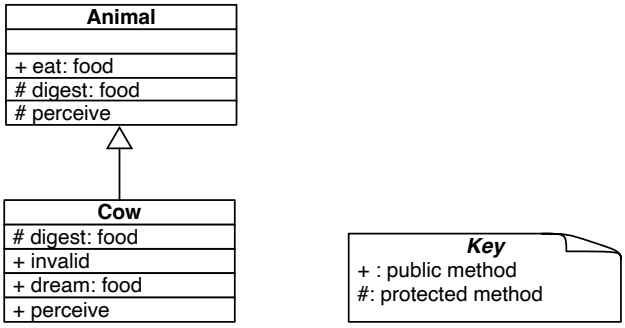

Figure 1: A sample hierarchy.

Consider also the following Pharo definitions for those classes, where protected methods are declared using the `<protected>` pragma:

```
1 Animal >> eat: food
2     ^ self digest: food
3 Animal >> digest: food
4     <protected>
5     ^ 'nom ',food.
```

```
6  Animal >> perceive
7      <protected>
8      ^ 'time goes by'
9
10 Cow >> digest: food
11     <protected>
12     ^ 'nom nom nom nom', food.
13 Cow >> invalid
14     ^ Animal new digest: 'apple'.
15 Cow >> dream: food
16     ^ 'I would like to ', (self eat: food).
17 Cow >> perceive
18     ^ 'I can see the sun shining and notice that ', super perceive.
19
20 "See below several scenarios one could test in the Playground where >>> is the expected output:"
21 anAnimal := Animal new. aCow := Cow new. anAnimal eat: 'grass'.
22 ">>> 'nom grass"
23 anAnimal digest: 'grass'.
24 ">>> anAnimal did not understand #digest:"
25 aCow eat: 'grass'.
26 ">>> 'nom nom nom nom grass'"
27 aCow invalid.
28 ">>> anAnimal did not understand #digest:"
29 aCow dream: 'grass'.
30 ">>> 'I would like to nom nom nom nom grass'"
31 aCow perceive.
32 ">>> 'I can see the sun shining and notice that time goes by'"
```

The protected modifier features the following properties:

- A protected method is visible from self-sends within a class (defined on lines 2 and 16, used on lines 22, 26 and 30).
- A protected method is visible from super-sends from subclasses (defined on line 18, used on line 32).
- A protected method is invisible from outside the class hierarchy, resulting in a runtime error (on line 24).
- A protected method is invisible from the defining class when it is neither a self-send nor a super-send (incorrect definition on line 14 resulting in runtime error on line 28).

The design choice to rely on self-sends and super-sends allows for compile-time visibility distinction, avoiding the need for a runtime mechanism that would use dynamic information. This "self-send-based visibility" design has been recently published in [2]. To implement it, you need to extend Pharo's compiler as follows.

First, you will support *double registration of public methods*. As mentioned before, developers identify protected methods using pragma's. Protected methods will be registered in the method dictionary of their class with a *mangled* ("hidden") selector. Generating a mangled selector happens by applying an injective function on the selector, for example, by prefixing it with __ like *__digest:*. Public methods, on the other hand, are going to appear twice in the method dictionary, once without prefix (e.g. *eat:*) and once with the prefix (e.g. *__eat:*). This is because they are visible both from self-sends and object-sends.

Second, during compilation, all self-sends are mangled. This means that syntactic occurrences of self-sends (including super-sends) are replaced to call the mangled selector.

Requirement 1 Extend the compiler to support compilation of `<protected>` methods, using *double registration of public methods* and *selector mangling of self-sends* following the aforementioned Thomas et al. [2]'s design.

In particular, your extension should perform the following at compile time:

- You must distinct the compilation of methods with the `<protected>` pragma and without.
- Associate the compilation result for each method with a mangled selector in the method dictionary.

- Implement *double registration of public methods* to ensure 'public' selector association with the method in the method dictionary.

- Implement *selector mangling of self-sends* to ensure each self-send (or super-sends) is replaced by a message send targetting the mangled selector.

Requirement 2 To ensure backward compatibility with existing code in the system, it is essential that the technique *selector mangling of self-sends* performs replacement for selectors that are certain to have a mangled selector registration. As superclasses without protected methods do not have double registration of public methods (e.g. `Object»#=`), self-sends will not find mangled entries in their method dictionaries. As such, you should ensure that self-sends are mangled only when a the current class or a superclass provides a `<protected>` implementation of the selector.

Note, a public method cannot be overridden as `<protected>` as this introduces ambiguity (as also reported in [2]); as such, this case should not be covered by your implementation.

2.2 Object-Centric Debugging

During debugging, developers often target a particular portion of the stack to debug, however usually it helps to narrow the debugging scope further down to a particular object. The goal of this extension is to enable a developer to place breakpoints on a specific part of a method, but further restrict this to a particular object. As a concrete example, consider the following code snippet.

```
1  aHuman := Human new.
2  anotherHuman := Human new.
3
4  breaker := ObjectBreaker for: aHuman.
5  breaker onMessageSend.                                          "(a)"
6  breaker onMessageSendIn: #greet.                                "(b)"
7  breaker onMessageSendIn: #greet when: [ :send | send selector = #+ ]. "(c)"
8
9  aHuman greet.       "(d)"
10 anotherHuman greet. "(e)"
11
12 breaker clear.      "(f)"
```

In this example, the developer creates an `ObjectBreaker` for a particular object (i.e., `aHuman`). The breaker is then configured to (a) break[1] on every message send, (b) on every message send in the method `greet` and (c) on every message send in `greet` where the message selector is +. When `aHuman` then sends a message (d) the debugger should then open. However, this should not affect debugging for other instances of Human (e) that are not the target of the breaker (i.e., `anotherHuman`). Finally, the breaker can be cleared to remove all breakpoints (f).

Supporting this kind of object-centric debugging has been recently proposed by Costiou et al. [1] as a reflection-based approach to support debugging of object-oriented programs. Your implementation may be inspired by the approach of Costiou et al. [1], but you are free to choose your own design and implementation. We detail the approach of Costiou et al. as a guideline below.

Their implementation relies on the creation of anonymous subclasses of the class of the target object (i.e., `aHuman`) and ensuring the particular instance becomes an instance of the newly created class. Figure 2 illustrates the approach for the case of code shown in the example above.

The figure illustrates the three main steps of the approach. First, the target object (i.e., `aHuman`) is selected. Second, an anonymous subclass of the class of the target object is created (i.e., `Human`). For this subclass the relevant methods (i.e., `greet`) is overridden, in which a breakpoint is added on the relevant condition Third, the target object is made an instance of the anonymous subclass, ensuring that the relevant breakpoints are hit when the target object sends messages.

[1] In this context, breaking means opening the debugger when the relevant condition is met.

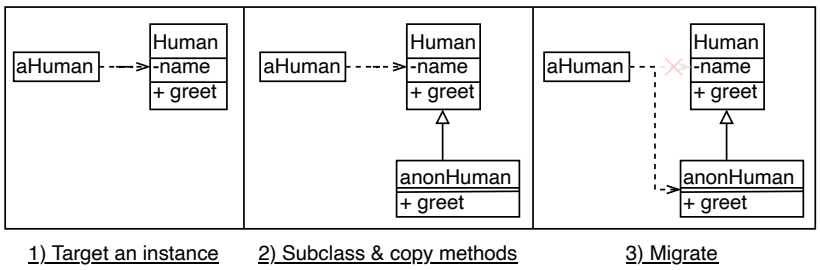

Figure 2: Object-centric debugging approach.

Note that the approach of Costiou et al. [1] further takes into account the case when the target methods perform super-sends. That would be problematic as copied-over super-sends will invoke the method in the original class, eg. `Human`, rather than the superclass of the the target class (e.g. the superclass of `Human`).

This case, however, is out of scope for this project, and you should thus scan the method to ensure no super-sends are present. Should super-sends be present in the target class, then your implementation should signal the error `ObjectCentricDebuggingDoesNotSupportSuperSends`.

Requirement Extend Pharo to support object-centric debugging based on the approach of Costiou et al. [1]. Your implementation should at least support the three configurations of the breaker in the example above (i.e., (a), (b) and (c)). Should the target class use super-sends, then your implementation should signal the error `ObjectCentricDebuggingDoesNotSupportSuperSends`.

2.3 Multiple exception handlers

The final extension focuses on adding multiple exception handlers to `BlockClosure`. The goal of this extension is to allow Pharo to have multiple exception handlers in a similar way to Java. When adding an exception handler in Pharo (using `on:do:`), a developer is currently not allowed to add different handlers for different type of exceptions, except for nesting `on:do:` blocks.

In this assignment, we would like to allow developers to add multiple exception handlers for block closures. As a concrete usage example consider the following code snippet.

```
block := [ :arg | arg / 0  ] asMyBlockClosure.
block handle: ZeroDivide with: [ :err | 'You divided by zero' ].
block handle: MessageNotUnderstood with: [ :err | 'I cannot divide nil' ].
res := block value: 1. >> 'You divided by zero'
res := block value: nil. >> 'I cannot divide nil'
```

Requirement 1. Extend `BlockClosure` to provide multiple exception handlers by manipulating execution stack, also leveraging on the exception handling mechanism of Pharo.

Requirement 2. The handlers should be called in order of specificity: if you add an handler for `Error` and one for `ZeroDivide` (that is a subclass of `Error`), a `ZeroDivide` should always call the `ZeroDivide` handler (if it is present), although the handler for `Error` is considered valid.

Requirement 3. The extension should not break the error handling through `on:do:`. This means that nesting `on:do:` blocks with `handle:with:` blocks should call the handlers in a correct order depending on their exception type.

Requirement 4. The final requirement is to add support for requesting the next handler, accessible through `callNextHandler`. When having two handlers for two subtypes (e.g., `ZeroDivide` and `Error`), in the handler of the subclass (i.e. `ZeroDivide`), when `self callNextHandler` is executed, the handler associated to the superclass (i.e. `Error`) is called.

3 Delivery

The deadline for delivering the project is **Friday, 8th of May 2026** at 23:59 (Brussels time). The project should be delivered via Canvas in the form of a unique `META-Project-[netid].zip` file where `[netid]` is your VUB netid (e.g., `META-Project-amunster.zip`). The deliverable should include:

The code of your solution. The implementation of the project should be in **Pharo Smalltalk version 12.0**. All the classes of your implementation should be in **a unique package** called `META-Project-[netid]`. If you add a method to an existent class (e.g., `BlockClosure`), make sure move that one as well to your package. (In the browser, right click on a method → *Refactorings* → *Move to package*)

The code to be delivered will consist of a **.st** file, that you can export *filing out* your package[2]. As your package, this file should be named `META-Project-[netid].st`.

Tests for your solution. Your deliverable should also include the implementation of the running (or testing) scenario that uses your extensions. Your should at least test the usage examples included in this assignment. The tests should be in the same package as the code of your solutiona in a subclass of `TestCase`.

A project report. Your deliverable should contain a report of **maximum 5 pages**. Such report must include instructions on how to load the package in a freshly created Pharo 12.0 image and execute the tests. The report further should detail your design choices and of the implementation.

Important: Please test your submission file by loading it in an empty image and running your testing code. In this way you will be sure that everything is correctly included in your submission!

4 Pharo Tips and Tricks

When working with Pharo at the level of AST, IR, and context manipulation, your code may cause the Pharo VM to crash. Hence, remember to **save your image** before you try new functionality. You can also use different tools that allow you to backup your code or recover lost changes:

Version control. You can use *iceberg*[3] to upload your code to a Git repository (Tools->Iceberg).

Recovering lost changes. You can use *epicea*[4] (Tools->Code Changes) to recover lost code changes.

5 Evaluation

This project accounts for 25% of your final grade. The project needs to be orally defended in order to obtain a mark. The oral defence will happen digitally on the scheduled day of the exam (check the exam schedule for the concrete date) as described in the "ExaminationInformation" document available on the Canvas course space. Implementation, report, and oral defence will all be taken into account for the evaluation.
Recall that you must hand in and defend every assignment in order to pass for the course as a whole.

Your project will be evaluated mostly on the correctness and functionality of your solution. The quality of your code and respect for the concepts of object-oriented programming are important and will be taken into account in the evaluation. The use of unit testing will also be considered in your mark.

Important: This project is to be made **strictly individually** and **on your own**. This means that you must create your [task/project] on an independent basis and you must be able to explain your work, reimplement it under supervision, and defend your solution. Copying work (code, text, etc.) from or sharing it with third parties (e.g., fellow students, websites, GitHub, generative AI tools, etc.) is not allowed. Electronic tools will be used to compare all submissions with online resources and with each other, even across academic years.

Any action by a student that deviates from the instructions given and does not comply with the examination regulations is considered an irregularity. In particular, any form of fraud that violates scientific integrity, including using statements or texts produced by generative AI tools without citing original sources or simulating or falsifying

[2] You can file out your package by right clicking on a package and selecting *Extra* → *File Out*.
[3] `https://github.com/pharo-vcs/iceberg` [4] `https://github.com/tinchodias/epicea`

research data, are irregularities (cf. OER, Article 118$2). Plagiarism is also an irregularity. Plagiarism means the use of other people's work, adapted or otherwise, without careful acknowledgement of sources. (cf. OER, Article 118$2). Plagiarism may relate to various forms of works, including text, image, music, data file, structure, train of thought, and ideas.

Any suspicion of plagiarism will be reported to the Dean of Faculty without delay. Both user and provider of such code will be reported and will be dealt with according to the plagiarism rules of the examination regulations (cf. OER, Article 118). The dean may decide on (a combination of) the disciplinary sanctions, ranging from 0/20 on the paper of the given programme unit to a prohibition from (re-)enrolling for one or multiple academic years (cf. OER, Article 118$5).

If you are in doubt as to whether or not something would be considered plagiarism, contact the lecturer or assistant.

Do not hesitate to contact me via e-mail (amunster@vub.be) if you have any questions.

Good luck!

References

[1] Steven Costiou, Vincent Aranega, and Marcus Denker. "Reflection as a tool to debug objects". In: *Proceedings of the 15th ACM SIGPLAN International Conference on Software Language Engineering*. 2022, pp. 55–60. URL: `https://inria.hal.science/hal-03846015/file/2022-Modern-Reflection-techniques-to-debug-object-hal-version.pdf`.

[2] Iona Thomas, Vincent Aranega, Stéphane Ducasse, Guillermo Polito, and Pablo Tesone. "A VM-Agnostic and Backwards Compatible Protected Modifier for Dynamically-Typed Languages". In: *The Art, Science, and Engineering of Programming* 8 (1 2024). URL: `https://arxiv.org/pdf/2306.12410v1`.